Bayesian Nonparametric Methods

Dirichlet Processes, Gaussian Processes, and Beyond

1 Introduction to Bayesian Inference

A general statistical analysis

- **Model the data.** Assign a probability model for $\boldsymbol{y} = (y_1, \ldots, y_n)$, e.g. supervised, unsupervised, or semi-supervised. *This choice is in general independent of the inference plan adopted next.*
- **Adopt an inference plan**: parameter estimation and/or hypothesis testing.

Let θ be a parameter of interest. We seek a function $\delta(\boldsymbol{y})$ of the data such that $\hat{\theta} = \delta(\boldsymbol{y})$ is a good estimate of θ.

Example 1.1 (Location-normal model)**.** Suppose

$$y_i = \theta + \varepsilon_i, \quad \varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2).$$

The maximum-likelihood estimator (MLE) is the sample mean:

$$\hat{\theta}_{\text{MLE}} = \bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i.$$

Example 1.2 (Bernoulli model)**.** Let $y_i \in \{0, 1\}$ be i.i.d. Bernoulli(θ) coin flips. The likelihood is

$$L(\theta \mid \boldsymbol{y}) = \theta^{\sum y_i} (1 - \theta)^{n - \sum y_i},$$

and the MLE is $\hat{\theta} = \bar{y}$ (proportion of heads).

Example 1.3 (Exponential model)**.** Let $y_i \overset{\text{iid}}{\sim} \text{Exp}(\theta)$, so $\mathbb{E}[y_i] = 1/\theta$. The MLE is $\hat{\theta} = 1/\bar{y}$.

The assumed distribution of the data y induces a distribution on $\hat{\theta} = \delta(\boldsymbol{y})$, since $\hat{\theta}$ is a function of the data. The frequentist goal is therefore to characterise this *sampling distribution* of $\hat{\theta}$, treating it as a proxy for the unknown θ, while also incorporating the information the data carries about θ.

Bayesians take a more direct route: rather than working with a distribution of an *estimator* of θ, they seek a distribution of θ *itself*. The natural entry point is the likelihood, $P(\boldsymbol{y} \mid \theta)$, which models the observed data conditional on the parameter. Applying Bayes' theorem inverts this conditioning, yielding

$$P(\theta \mid \boldsymbol{y}) \;\propto\; P(\theta)\, P(\boldsymbol{y} \mid \theta),$$

so the posterior $P(\theta \mid \boldsymbol{y})$ is precisely the distributional summary that directly characterises θ while incorporating the evidence from the data.

Bayesian strategy: Apply Bayes' theorem to obtain the *posterior distribution*

$$\boxed{P(\theta \mid \boldsymbol{y}) = \frac{P(\theta)\, P(\boldsymbol{y} \mid \theta)}{P(\boldsymbol{y})} \;\propto\; P(\theta)\, P(\boldsymbol{y} \mid \theta),}$$

where $P(\boldsymbol{y}) = \int P(\theta)\, P(\boldsymbol{y} \mid \theta)\, \mathrm{d}\theta$ is the marginal likelihood (normalising constant), often called the *evidence*. Thus, $P(\boldsymbol{y})$ does not contain any information of θ.

Hence, the first piece of information we need is $P(y \mid \theta)$, which is given by the assumed model (and it is a *requirement* for Bayesian inference to have a likelihood or generative model for the data, unlike in the frequentist framework).

The second is $P(\theta)$, which is called the prior for θ. It is termed "prior" because it represents information external to the data and, philosophically, should exist even before any data are observed. However, it also serves as a technical component that allows Bayes' theorem to operate.

Importantly, we have flexibility in its choice. Bayesian inference does not force one to specify a highly informative prior; one may choose a non-informative or weakly informative prior, in which case the posterior is driven almost entirely by the likelihood. The Bayesian nonparametric is also another option to make this choice more flexible.

Term	**Name**	**Role**
$P(\theta)$	Prior	Beliefs about θ before data
$P(\boldsymbol{y} \mid \theta)$	Likelihood	Data probability given θ
$P(\theta \mid \boldsymbol{y})$	Posterior	Updated beliefs after data

Table 1: The three Bayesian quantities.

Example 1.4 (Beta-Binomial conjugate model)**.** Suppose $y_i \overset{\text{iid}}{\sim} \text{Bernoulli}(\theta)$ and we place a $\text{Beta}(\alpha, \beta)$ prior on θ. With n flips and $k = \sum y_i$ heads, the posterior is

$$\theta \mid \boldsymbol{y} \;\sim\; \text{Beta}(\alpha + k,\; \beta + n - k).$$

For $\alpha = \beta = 2$ (a mild symmetric prior), $n = 10$, $k = 7$:

$$\theta \mid \boldsymbol{y} \;\sim\; \text{Beta}(9, 5), \qquad \mathbb{E}[\theta \mid \boldsymbol{y}] = \frac{9}{14} \approx 0.643.$$

This shrinks the MLE $\hat{\theta} = 0.7$ toward the prior mean 0.5.

The Likelihood Principle [Berger and Wolpert, 1988]:

1. All evidence about θ from an experiment is contained in the likelihood function $L(\theta) = P(\boldsymbol{y} \mid \theta)$.
2. Two likelihood functions $L_1(\theta) = c\, L_2(\theta)$ (proportional, c free of θ) contain the same information about θ.

Example 1.5 (Stopping-rule irrelevance)**.** An experimenter observes $k = 7$ heads in $n = 10$ trials.

Fixed-n design Flip exactly 10 times. $L_A(\theta) = \binom{10}{7}\theta^7(1-\theta)^3$.

Negative-binomial design Flip until 3 tails appear and it happens at the 10-*th* flip. $L_B(\theta) = \binom{9}{2}\theta^7(1-\theta)^3$.

Since $L_A(\theta)/L_B(\theta) = \binom{10}{7}/\binom{9}{2} = 120/36$ is free of θ, the two likelihoods are proportional. By the Likelihood Principle, they contain identical evidence about θ, yielding the same Bayesian posterior. The frequentist p-value, however, differs between designs, a violation of the Likelihood Principle.

2 What Is Bayesian Nonparametrics?

Bayesian nonparametrics (BNP) is the branch of Bayesian statistics in which the parameter is not a finite-dimensional vector but an infinite-dimensional object: a probability distribution, a regression function, a clustering partition, or a feature set. So, basically, we do not specifically assume any parametric distribution as prior and in some cases even the likelihood may be based on distributions with all infinite dimensional parameters.

In classical Bayes we place a prior on $\theta \in \mathbb{R}^p$, a vector of fixed dimension p chosen before seeing the data. In BNP we place a prior directly on objects such as

- a probability distribution $G(\cdot)$,
- a regression function $f(\cdot)$,
- a partition of the data into clusters, or
- a (potentially infinite) set of latent features.

The guiding philosophy is that model complexity should be determined by the data rather than fixed in advance. For a regression function f, the parameter of interest is $f(x)$ for *every* x in the domain which is an infinite-dimensional object. This is why nonparametric estimation requires additional structure: one must restrict or regularise the space of admissible functions, typically by placing a stochastic process prior on f.

3 The BNP Toolbox: What Objects Can Be Random?

Table 2 summarises the main BNP priors, organised by the type of random object they define.

Table 2: Common BNP priors and the objects they randomise.

Random object	**Prior process**
Probability distribution G	Dirichlet process
Regression function $f(x)$	Gaussian process
Infinite latent feature matrix	Beta process / Indian buffet process
Clustering with power-law growth	Pitman-Yor process
Survival or hazard function	Gamma process

4 Stochastic Processes: A Brief Introduction

A stochastic process is a collection of random variables indexed by some set, typically representing time or space. Formally, a stochastic process is a family

$$\{X_t : t \in \mathcal{T}\},$$

where each X_t is a random variable defined on a common probability space $(\Omega, \mathcal{F}, P)$ and $\mathcal{T}$ is the index set. Depending on the application, $\mathcal{T}$ may be discrete ($\mathcal{T} = \mathbb{N}$), continuous ($\mathcal{T} = \mathbb{R}_+$), or multi-dimensional ($\mathcal{T} = \mathbb{R}^d$).

Rather than asking about the distribution of a single random variable, the object of interest is the *joint* distribution of the entire family $(X_{t_1}, \ldots, X_{t_n})$ for any finite collection of indices

$t_1, \ldots, t_n \in \mathcal{T}$. A stochastic process is therefore a probability distribution over *functions* $\omega : \mathcal{T} \to \mathcal{S}$, where $\mathcal{S}$ is the state space.

4.1 Key Examples

Bernoulli process. The simplest discrete-time process: $X_1, X_2, \ldots$ are i.i.d. Bernoulli(p). Each $X_t \in \{0, 1\}$ records a success or failure at time t. Despite its simplicity it is the building block for the binomial distribution and, in the limit, the Poisson process.

Random walk. Let $\xi_1, \xi_2, \ldots$ be i.i.d. with mean zero and set $S_0 = 0$, $S_n = \sum_{i=1}^{n} \xi_i$. The sequence $(S_n)_{n \geq 0}$ is a random walk. Its paths wander without a preferred direction, and increments over disjoint intervals are independent.

```
set.seed(42)
n <- 200
p <- 0.5

# Bernoulli process
xi <- rbinom(n, size = 1, prob = p)

# Random walk: cumulative sum of +1/-1 increments
increments <- ifelse(xi == 1, 1, -1)
S <- c(0, cumsum(increments))

par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))

# Bernoulli path
plot(seq_len(n), xi, type = "h", lwd = 0.8,
     col  = "steelblue",
     xlab = "", ylab = expression(X[t]),
     main = "Bernoulli process  (p = 0.5)",
     yaxt = "n")
axis(2, at = c(0, 1))

# Random walk path
plot(0:n, S, type = "l", lwd = 1.2,
     col  = "steelblue",
     xlab = "t", ylab = expression(S[n]),
     main = "Symmetric random walk")
abline(h = 0, lty = 2, col = "grey70")

par(mfrow = c(1, 1))
```

Poisson process. A continuous-time process $\{N_t : t \geq 0\}$ counting the number of events in $[0, t]$. It is characterised by three properties: $N_0 = 0$; increments over disjoint intervals are independent, and the number of events in any interval of length ℓ follows Poisson($\lambda \ell$) for some rate $\lambda > 0$.

One can simulate a Poisson process by exploiting the order-statistic property directly: given $N(T_{\max}) = n$, the n arrival times are the order statistics of n i.i.d. Uniform$(0, T_{\max})$ draws. The

algorithm is:

1. Draw the total count: $n \sim \text{Poisson}(\lambda T_{\max})$.
2. Draw n arrival times uniformly: $U_1, \ldots, U_n \overset{\text{iid}}{\sim} \text{Uniform}(0, T_{\max})$.
3. Sort to obtain the ordered epochs $\tau_1 < \cdots < \tau_n$.
4. Reconstruct $N(t)$ as a step function.

```
set.seed(7)

sim_poisson_process <- function(lambda, T_max) {
  times  <- numeric(0)
  t      <- 0
  repeat {
    t <- t + rexp(1, rate = lambda)
    if (t > T_max) break
    times <- c(times, t)
  }
  times
}

T_max   <- 20
lambdas <- c(0.5, 1, 3)
cols    <- c("steelblue", "firebrick", "darkgreen")

plot(NA, xlim = c(0, T_max), ylim = c(0, T_max * max(lambdas)),
     xlab = "Time t", ylab = "N(t)",
     main = "Poisson process sample paths")

for (j in seq_along(lambdas)) {
  arrivals <- sim_poisson_process(lambdas[j], T_max)
  N        <- seq_along(arrivals)
  t_plot   <- c(0, rep(arrivals, each = 2), T_max)
  N_plot   <- c(0, 0, rep(N, each = 2))
  lines(t_plot, N_plot, col = cols[j], lwd = 1.8)
}

legend("topleft", bty = "n",
       legend = paste0("lambda = ", lambdas),
       col = cols, lwd = 2)
```

There is another approach that makes no direct use of the Poisson distribution. Instead, it exploits the following characterization: generate exponential inter-arrival times and accumulate them into arrival epochs until the observation window is exhausted. Concretely, the algorithm is:

1. Set $\tau_0 = 0$.
2. Draw $T_k \sim \text{Exp}(\lambda)$ independently.

3. Set $\tau_k = \tau_{k-1} + T_k$.

4. If $\tau_k > T_{\max}$, stop. Otherwise record τ_k as an arrival and return to step 2.

5. Return the counting process $N(t) = \#\{k : \tau_k \le t\}$.

This is precisely the `repeat` loop in the code: `rexp(1, rate = lambda)` draws T_k, the accumulated `t` is τ_k, and `times` collects the arrival epochs.

```
set.seed(7)

sim_poisson_rpois <- function(lambda, T_max) {
  # Step 1: draw total number of arrivals in [0, T_max]
  n <- rpois(1, lambda = lambda * T_max)
  if (n == 0) return(numeric(0))
  # Step 2-3: arrival times = order statistics of Uniform(0, T_max)
  sort(runif(n, min = 0, max = T_max))
}

T_max   <- 20
lambdas <- c(0.5, 1, 3)
cols    <- c("steelblue", "firebrick", "darkgreen")

plot(NA, xlim = c(0, T_max), ylim = c(0, T_max * max(lambdas)),
     xlab = "Time t", ylab = "N(t)",
     main = "Poisson process sample paths (via rpois)")

for (j in seq_along(lambdas)) {
  arrivals <- sim_poisson_rpois(lambdas[j], T_max)
  N        <- seq_along(arrivals)
  t_plot   <- c(0, rep(arrivals, each = 2), T_max)
  N_plot   <- c(0, 0, rep(N, each = 2))
  lines(t_plot, N_plot, col = cols[j], lwd = 1.8)
}

legend("topleft", bty = "n",
       legend = paste0("lambda = ", lambdas),
       col = cols, lwd = 2)
```

The plotting code reconstructs $N(t)$ as a right-continuous step function. Given arrival epochs $\tau_1 < \tau_2 < \cdots < \tau_n \le T_{\max}$, the sample path is

$$N(t) = \sum_{k=1}^{n} \mathbf{1}[\tau_k \le t], \quad t \in [0, T_{\max}],$$

which equals k on the half-open interval $[\tau_k, \tau_{k+1})$ and jumps by exactly 1 at each τ_k. The `rep(..., each = 2)` construction in the code creates the horizontal and vertical segments of this step function explicitly.

Although `rpois` is never called, the Poisson distribution emerges as a theorem. The total number of arrivals recorded by the simulation is

$$N(T_{\max}) = \max\{k : \tau_k \le T_{\max}\} \sim \text{Poisson}(\lambda\, T_{\max}),$$

with mean and variance $\lambda\, T_{\max}$. For the three rates $\lambda \in \{0.5, 1, 3\}$ and $T_{\max} = 20$, the expected final counts are 10, 20, and 60 respectively – consistent with the step-function heights in the plot. The Poisson distribution is not an assumption here; it is a consequence of accumulating i.i.d. exponential inter-arrival times.

Brownian motion. A continuous-time process $\{W_t : t \ge 0\}$ with $W_0 = 0$, independent increments, and $W_t - W_s \sim \text{Normal}(0, t - s)$ for $t > s$. Brownian motion has continuous sample paths almost surely and is the continuous limit of the random walk. It is the canonical building block of continuous-time probability and the foundation for the Gaussian process.

```
set.seed(2024)

sim_bm <- function(n, T_max = 1) {
  dt <- T_max / n
  dW <- rnorm(n, mean = 0, sd = sqrt(dt))
  W  <- c(0, cumsum(dW))
  t  <- seq(0, T_max, length.out = n + 1)
  list(t = t, W = W)
}

n_paths <- 8
T_max   <- 1
n_steps <- 500

plot(NA, xlim = c(0, T_max), ylim = c(-2.5, 2.5),
     xlab = "t", ylab = expression(W[t]),
     main = "Brownian motion sample paths")
abline(h = 0, lty = 2, col = "grey80")

for (i in seq_len(n_paths)) {
  bm <- sim_bm(n_steps, T_max)
  lines(bm$t, bm$W,
        col = adjustcolor("steelblue", alpha.f = 0.55),
        lwd = 1.2)
}

t_seq <- seq(0, T_max, length.out = 300)
lines(t_seq,  1.96 * sqrt(t_seq), col = "firebrick",
      lwd = 1.5, lty = 2)
lines(t_seq, -1.96 * sqrt(t_seq), col = "firebrick",
      lwd = 1.5, lty = 2)
legend("topleft", bty = "n",
       legend = c("Sample paths", "95% band"),
```

```
col    = c("steelblue", "firebrick"),
lty    = c(1, 2), lwd = 2)
```

The band comes from the marginal distribution of W_t at each fixed t. Since $W_t \sim \text{Normal}(0, t)$, a pointwise 95% interval at each t is

$$W_t \in \left[-1.96\sqrt{t},\ 1.96\sqrt{t}\right],$$

because $P(-1.96 \le Z \le 1.96) = 0.95$ for $Z \sim \text{Normal}(0, 1)$, and $W_t = \sqrt{t}\, Z$.

Derivation. Brownian motion has independent increments with

$$W_t - W_s \sim \text{Normal}(0,\ t - s), \quad t > s \ge 0,$$

and $W_0 = 0$. Setting $s = 0$ gives

$$W_t \sim \text{Normal}(0,\ t), \qquad \text{Standard} - \text{deviation}(W_t) = \sqrt{t}.$$

The pointwise 95% interval is therefore

$$\left[\mu \pm 1.96\,\sigma\right] = \left[0 \pm 1.96\sqrt{t}\right],$$

which is exactly what the code draws via `1.96 * sqrt(t_seq)`. The band opens as $\sqrt{t}$, reflecting the diffusive growth of uncertainty: the process wanders further from zero on average as time increases, but at a decelerating rate.

What the band is and is not.

- It is a *pointwise* band: at each fixed t, 95% of the marginal probability mass of W_t lies within $[-1.96\sqrt{t},\ 1.96\sqrt{t}]$.
- It is *not* a simultaneous band: the probability that an entire sample path stays within the band for *all* $t \in [0, T_{\max}]$ simultaneously is strictly less than 0.95. By the reflection principle, the probability that Brownian motion ever exits a symmetric interval $[-c, c]$ on $[0, T]$ is

  $$P\left(\sup_{0 \le t \le T} |W_t| > c\right) = 2\,P(W_T > c) + (\text{correction terms}),$$

  which can be substantially larger than 5% even for moderate c. A simultaneous 95% band would need to be considerably wider.

Markov chain. A discrete- or continuous-time process satisfying the *Markov property*: given the present state, the future is independent of the past,

$$P(X_{t+1} = x \mid X_0, \ldots, X_t) = P(X_{t+1} = x \mid X_t).$$

A finite, discrete-time Markov chain on states $\mathcal{S} = \{s_1, \ldots, s_m\}$ is fully characterised by its *transition matrix* $\boldsymbol{P} \in [0, 1]^{m \times m}$, where $P_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$ and each row sums to one. The chain is *time-homogeneous* if $\boldsymbol{P}$ does not depend on t.

A distribution $\boldsymbol{\pi}$ is *stationary* for $\boldsymbol{P}$ if $\boldsymbol{\pi}^\top \boldsymbol{P} = \boldsymbol{\pi}^\top$, i.e. $\boldsymbol{\pi}$ is a left eigenvector of $\boldsymbol{P}$ corresponding to eigenvalue 1. Under mild regularity conditions (irreducibility and aperiodicity), the chain converges to $\boldsymbol{\pi}$ from any starting state:

$$P(X_t = s_j \mid X_0 = s_i) \;\to\; \pi_j \quad \text{as } t \to \infty.$$

This convergence property is exploited directly in MCMC: one constructs a transition kernel $\boldsymbol{P}$ whose stationary distribution is the target posterior $p(\theta \mid \boldsymbol{y})$, then runs the chain long enough for the empirical distribution of its states to approximate the posterior.

The code below simulates a three-state chain with transition matrix

$$\boldsymbol{P} = \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.3 & 0.4 & 0.3 \\ 0.1 & 0.3 & 0.6 \end{pmatrix},$$

computes the stationary distribution as the normalised left eigenvector of $\boldsymbol{P}$ for eigenvalue 1, simulates $n = 2000$ steps, and plots both the sample path and the convergence of the empirical state frequencies to $\boldsymbol{\pi}$.

```
set.seed(99)

P <- matrix(c(0.7, 0.2, 0.1,
              0.3, 0.4, 0.3,
              0.1, 0.3, 0.6),
            nrow = 3, byrow = TRUE)

# Stationary distribution: normalised left eigenvector for
# eigenvalue 1, i.e. solution to pi^T P = pi^T, sum(pi) = 1
eig     <- eigen(t(P))
pi_stat <- Re(eig$vectors[, 1])
pi_stat <- pi_stat / sum(pi_stat)
cat("Stationary distribution:", round(pi_stat, 4), "\n")

# Simulate the chain by drawing from the row of P
# corresponding to the current state
sim_mc <- function(P, n, state0 = 1) {
  states    <- integer(n)
  states[1] <- state0
  for (t in 2:n)
    states[t] <- sample(nrow(P), 1, prob = P[states[t-1], ])
  states
}

n     <- 2000
chain <- sim_mc(P, n, state0 = 1)
cols  <- c("steelblue", "firebrick", "darkgreen")

par(mfrow = c(1, 2))
```

```
# Left panel: sample path for first 200 steps
plot(seq_len(200), chain[1:200], type = "s",
     col  = "steelblue", lwd = 1,
     xlab = "t", ylab = "State",
     main = "Markov chain path (first 200 steps)",
     yaxt = "n")
axis(2, at = 1:3, labels = paste0("s", 1:3))

# Right panel: empirical frequencies vs stationary distribution
# freq[k, m] = proportion of first m steps spent in state k
freq <- sapply(seq(10, n, by = 10), function(m)
  tabulate(chain[1:m], nbins = 3) / m)

matplot(seq(10, n, by = 10), t(freq),
        type = "l", lty = 1, lwd = 1.5, col = cols,
        xlab = "n", ylab = "Empirical frequency",
        main = "Convergence to stationary distribution")

# Overlay true stationary probabilities as horizontal dashed lines
for (k in 1:3)
  abline(h = pi_stat[k], col = cols[k], lty = 2, lwd = 1)

legend("right", bty = "n",
       legend = paste0("State ", 1:3),
       col = cols, lwd = 2)

par(mfrow = c(1, 1))
```

Remark 4.1. The left panel illustrates the Markov property directly: transitions depend only on the current state, not on the history, so the path switches between states according to the row of $\boldsymbol{P}$ corresponding to where the chain currently sits. The right panel illustrates ergodicity: the empirical frequency of each state converges to its stationary probability π_k as $n \to \infty$, regardless of the starting state s_1. The rate of convergence is governed by the *spectral gap* $1 - |\lambda_2|$, where λ_2 is the second-largest eigenvalue of $\boldsymbol{P}$ in absolute value; a larger gap means faster mixing.

4.2 Characterising a Stochastic Process

A stochastic process is fully characterised by its *finite-dimensional distributions* (fdds): the collection of joint distributions

$$\left(X_{t_1}, \ldots, X_{t_n}\right)$$

for all $n \geq 1$ and all choices of $t_1, \ldots, t_n \in \mathcal{T}$. By *Kolmogorov's extension theorem*, any consistent family of fdds, one that is symmetric under permutation and compatible under marginalisation, is the fdd family of some stochastic process. This theorem justifies defining complex processes, including the Gaussian process and the Dirichlet process, through their finite-dimensional marginals alone.

4.3 Two Important Properties

Stationarity. A process is *(strictly) stationary* if the joint distribution of $(X_{t_1}, \ldots, X_{t_n})$ is invariant under time shifts: for all h,

$$(X_{t_1+h}, \ldots, X_{t_n+h}) \overset{d}{=} (X_{t_1}, \ldots, X_{t_n}).$$

A weaker condition, *second-order stationarity*, requires only that the mean $\mathbb{E}[X_t]$ is constant and the covariance $\mathrm{Cov}(X_t, X_s)$ depends only on the lag $|t - s|$. Most standard kernels used in GP regression encode second-order stationarity.

Exchangeability. A sequence $(X_1, X_2, \ldots)$ is *exchangeable* if its joint distribution is invariant under finite permutations:

$$(X_1, \ldots, X_n) \overset{d}{=} (X_{\sigma(1)}, \ldots, X_{\sigma(n)})$$

for every permutation σ and every n. By *de Finetti's theorem*, every exchangeable sequence is a mixture of i.i.d. sequences:

$$X_1, X_2, \ldots \mid G \overset{\text{iid}}{\sim} G, \qquad G \sim \Pi.$$

This theorem is the theoretical foundation of Bayesian nonparametrics: the random measure G is precisely the infinite-dimensional parameter over which a BNP prior is placed. The Dirichlet process, the Pitman-Yor process, and the beta process all arise as the de Finetti mixing measure for natural exchangeable processes on partitions, sequences, and binary matrices respectively.

4.4 From Stochastic Processes to BNP

The connection between stochastic processes and Bayesian nonparametrics is direct. A BNP prior is simply a probability distribution over an infinite-dimensional parameter space, and such a distribution is, by definition, a stochastic process.

- A **Gaussian process** $f \sim \mathrm{GP}(m, k)$ is a stochastic process indexed by $\mathcal{X}$, with every finite collection of function values jointly Gaussian. It is a prior over regression functions.
- A **Dirichlet process** $G \sim \mathrm{DP}(\alpha, G_0)$ is a stochastic process indexed by measurable sets, with every finite partition of the sample space yielding a Dirichlet-distributed vector. It is a prior over probability measures.
- The **IBP** and **Pitman-Yor process** are likewise stochastic processes, on binary matrices and probability measures respectively, whose finite-dimensional distributions are specified sequentially via the buffet and two-parameter CRP constructions.

Kolmogorov's extension theorem guarantees that specifying consistent finite-dimensional distributions is sufficient to define each of these processes rigorously, even though the full parameter space is infinite-dimensional.

4.4.1 Gaussian Process Sample Paths

```
set.seed(11)

k_se <- function(x1, x2, ell = 1)
  exp(-(outer(x1, x2, "-"))^2 / (2 * ell^2))
```

```
k_mat <- function(x1, x2, ell = 1) {
  r <- abs(outer(x1, x2, "-"))
  (1 + sqrt(5)*r/ell + 5*r^2/(3*ell^2)) * exp(-sqrt(5)*r/ell)
}

k_per <- function(x1, x2, ell = 1, p = 1)
  exp(-2 * sin(pi * abs(outer(x1, x2, "-")) / p)^2 / ell^2)

draw_paths <- function(kernel, x, n_samps = 6, jitter = 1e-8) {
  K <- kernel(x, x) + jitter * diag(length(x))
  L <- t(chol(K))
  L %*% matrix(rnorm(length(x) * n_samps), length(x))
}

x       <- seq(0, 4, length.out = 300)
n_samps <- 6
kernels <- list(
  "SE  (ell = 1)"        = function(a, b) k_se(a,  b, ell = 1),
  "Matern-5/2  (ell = 1)" = function(a, b) k_mat(a, b, ell = 1),
  "Periodic  (p = 1)"    = function(a, b) k_per(a, b, ell = 1, p = 1)
)

par(mfrow = c(1, 3), mar = c(4, 3, 3, 1))
for (nm in names(kernels)) {
  paths <- draw_paths(kernels[[nm]], x, n_samps)
  matplot(x, paths, type = "l", lty = 1, lwd = 1.2,
          col  = adjustcolor(1:n_samps, 0.65),
          xlab = "x", ylab = "f(x)", main = nm,
          ylim = c(-3, 3))
  abline(h = 0, lty = 2, col = "grey70")
}
par(mfrow = c(1, 1))
```

Remark 4.2. The three panels make the kernel's role concrete. The SE kernel produces infinitely differentiable paths; the Matérn-5/2 kernel produces paths that are twice differentiable but visibly rougher; the periodic kernel produces paths that repeat exactly with period $p = 1$. Choosing a kernel is therefore equivalent to choosing a prior belief about the regularity of the unknown function.

5 Dirichlet Process: Random Distributions and Clustering

Before introducing the Dirichlet process itself, it helps to build up from the finite case: the Dirichlet distribution.

5.1 The Dirichlet Distribution

Let K be a fixed number of categories. A random vector $(\pi_1, \ldots, \pi_K)$ follows a Dirichlet distribution,

$$(\pi_1, \ldots, \pi_K) \sim \mathrm{Dir}(\alpha_1, \ldots, \alpha_K),$$

if its density with respect to Lebesgue measure on the $(K-1)$-simplex $\Delta^{K-1} = \{\boldsymbol{\pi} : \pi_k \geq 0, \sum_k \pi_k = 1\}$ is

$$p(\pi_1, \ldots, \pi_K) = \frac{\Gamma\left(\sum_{k=1}^K \alpha_k\right)}{\prod_{k=1}^K \Gamma(\alpha_k)} \prod_{k=1}^K \pi_k^{\alpha_k - 1}.$$

The parameters $\alpha_k > 0$ are *concentration parameters*; they control how much mass is placed near each corner of the simplex.

Symmetric Dirichlet. Setting $\alpha_k = \alpha/K$ for all k gives the symmetric case. Three regimes are instructive:

- $\alpha/K \gg 1$: draws are nearly uniform; all K categories receive similar weight.
- $\alpha/K = 1$: the uniform distribution over the simplex.
- $\alpha/K \ll 1$: draws are sparse; most mass concentrates on one or two categories.

Conjugacy. The Dirichlet is conjugate to the Categorical/Multinomial likelihood. Suppose we observe n i.i.d. draws $z_1, \ldots, z_n$ from a categorical model:

$$\boldsymbol{\pi} \sim \mathrm{Dir}(\alpha_1, \ldots, \alpha_K), \qquad z_i \mid \boldsymbol{\pi} \overset{\text{iid}}{\sim} \mathrm{Categorical}(\boldsymbol{\pi}), \quad i = 1, \ldots, n.$$

Let $n_k = \#\{i : z_i = k\}$ be the number of observations in category k. The posterior is

$$\boldsymbol{\pi} \mid z_{1:n} \;\sim\; \mathrm{Dir}(\alpha_1 + n_1,\; \ldots,\; \alpha_K + n_K).$$

Observing n_k counts in category k simply adds n_k to the corresponding concentration parameter, an elegant closed-form update.

Marginal (Pólya urn). Integrating out $\boldsymbol{\pi}$ gives the *Pólya urn* predictive distribution for the $(n+1)$-th observation given the first n:

$$P(z_{n+1} = k \mid z_{1:n}) = \frac{\alpha_k + n_k}{\sum_j \alpha_j + n},$$

a formula that foreshadows the Chinese Restaurant Process below.

5.2 From Finite Mixtures to Infinite Mixtures

A finite Bayesian mixture model with K components places a Dirichlet prior on the mixing weights:

$$\boldsymbol{\pi} \sim \mathrm{Dir}\left(\tfrac{\alpha}{K}, \ldots, \tfrac{\alpha}{K}\right), \qquad \phi_k \overset{\text{iid}}{\sim} G_0, \qquad z_i \mid \boldsymbol{\pi} \sim \mathrm{Categorical}(\boldsymbol{\pi}), \qquad x_i \mid z_i \sim p(x \mid \phi_{z_i}).$$

For finite K we must choose the number of components in advance. A natural question is: *what happens as $K \to \infty$?*

As $K \to \infty$ with α held fixed, the symmetric $\mathrm{Dir}(\alpha/K, \ldots, \alpha/K)$ prior converges (in a distributional sense on the space of probability measures) to the *Dirichlet process* with concentration α and base measure G_0. The Dirichlet process is therefore the natural nonparametric limit of the finite Dirichlet mixture.

5.3 The Dirichlet Process

Definition 5.1 (Dirichlet process)**.** A random probability measure G on a measurable space $(\Theta, \mathcal{B})$ is a *Dirichlet process* with concentration parameter $\alpha > 0$ and base measure G_0, written $G \sim \mathrm{DP}(\alpha, G_0)$, if for every finite measurable partition $(A_1, \ldots, A_K)$ of Θ,

$$\big(G(A_1), \ldots, G(A_K)\big) \;\sim\; \mathrm{Dir}\big(\alpha G_0(A_1), \ldots, \alpha G_0(A_K)\big).$$

The two parameters play distinct roles:

- G_0 is the *base measure*: $\mathbb{E}[G(A)] = G_0(A)$ for every measurable set A, so G_0 is the prior guess at the shape of G.
- α is the *concentration*: larger α pulls G closer to G_0; smaller α allows G to deviate more. Formally, $\mathrm{Var}[G(A)] = G_0(A)(1 - G_0(A))/(\alpha + 1)$, so variance shrinks as $\alpha \to \infty$.

5.4 Key Property: Almost Sure Discreteness

A draw $G \sim \mathrm{DP}(\alpha, G_0)$ is *almost surely discrete*, regardless of whether G_0 itself is continuous. The *stick-breaking construction* [Sethuraman, 1994] makes this explicit. Draw

$$v_k \overset{\text{iid}}{\sim} \mathrm{Beta}(1, \alpha), \qquad \phi_k \overset{\text{iid}}{\sim} G_0,$$

and set

$$\pi_k = v_k \prod_{j=1}^{k-1} (1 - v_j).$$

Then

$$G = \sum_{k=1}^{\infty} \pi_k \, \delta_{\phi_k} \;\sim\; \mathrm{DP}(\alpha, G_0).$$

The weights (π_k) partition the unit interval: at each step a fraction v_k of the remaining stick is broken off. Large α produces many small pieces; small α concentrates mass on the first few atoms.

What does $G = \sum_{k=1}^{\infty} \pi_k \, \delta_{\phi_k}$ mean? The expression defines G as a countably infinite weighted sum of point masses. To draw a sample $y \sim G$ means to draw from a categorical distribution with countably infinite support: $P(y = \phi_k) = \pi_k$ for $k = 1, 2, \ldots$ In practice one selects atom ϕ_k with probability π_k, so G assigns all of its mass to the discrete set $\{\phi_1, \phi_2, \ldots\}$ even when the base measure G_0 is continuous.

Consequence: ties and clustering. Because G is discrete, draws $\theta_1, \ldots, \theta_n \mid G \overset{\text{iid}}{\sim} G$ will coincide with positive probability: $P(\theta_i = \theta_j) > 0$ for $i \neq j$. These ties partition the indices $\{1, \ldots, n\}$ into groups sharing a common atom, an automatic clustering induced by the prior.

5.5 Chinese Restaurant Process

The CRP is the marginal process obtained by integrating out G. It describes the predictive distribution of cluster assignments directly, without ever instantiating G.

Suppose we observe n data points and let $z_i \in \{1, 2, 3, \ldots\}$ denote the cluster label of observation i, defined by

$$z_i = k \iff \theta_i = \phi_k,$$

where ϕ_k is the k-th distinct atom of G. Observations i and j belong to the same cluster whenever $z_i = z_j$. Let $K_n = |\{z_1, \dots, z_n\}|$ denote the number of distinct clusters among the first n observations, and let $n_k = \#\{i \le n : z_i = k\}$ denote the size of cluster k after n observations, so that $\sum_{k=1}^{K_n} n_k = n$.

The CRP gives the predictive distribution of z_{n+1} given the previous assignments $z_{1:n} = (z_1, \dots, z_n)$ after marginalising out G. It has a natural sequential description: imagine n customers already seated in a restaurant with infinitely many tables, each table corresponding to one cluster. Customer $n+1$ sits at:

$$P(z_{n+1} = k \mid z_{1:n}) = \begin{cases} \dfrac{n_k}{n+\alpha}, & \text{table } k \text{ already occupied } (n_k > 0), \\ \dfrac{\alpha}{n+\alpha}, & \text{a new, unoccupied table,} \end{cases}$$

where the probabilities sum to one since $\sum_{k=1}^{K_n} n_k + \alpha = n + \alpha$. The expected number of occupied tables after n customers grows as $\mathbb{E}[K_n] \approx \alpha \log n$.

Two properties are worth highlighting:

- *Rich get richer.* Popular tables attract new customers in proportion to their current size n_k, producing a heavy-tailed cluster-size distribution.
- *Exchangeability.* The joint distribution of $(z_1, \dots, z_n)$ under the CRP is exchangeable (de Finetti), which by the DP representation theorem is exactly what one expects from i.i.d. draws from a DP-distributed G.

5.6 Dirichlet Process Mixture Model

The full generative model combines the DP prior with a parametric emission distribution:

$$\begin{aligned} G &\sim \mathrm{DP}(\alpha, G_0), \\ \theta_i \mid G &\overset{\text{iid}}{\sim} G, \quad i = 1, \dots, n, \\ x_i \mid \theta_i &\sim p(x_i \mid \theta_i). \end{aligned}$$

Because G is discrete, many θ_i coincide; observations sharing the same atom ϕ_k form a mixture component governed by $p(x \mid \phi_k)$. Equivalently, introducing cluster labels z_i as above, the model can be written as

$$\begin{aligned} z_i \mid \boldsymbol{\pi} &\sim \mathrm{Categorical}(\boldsymbol{\pi}), \\ x_i \mid z_i &\sim p(x_i \mid \phi_{z_i}), \end{aligned}$$

where $\boldsymbol{\pi} = (\pi_1, \pi_2, \dots)$ are the stick-breaking weights and $\phi_k \overset{\text{iid}}{\sim} G_0$ are the component parameters. Marginalising over G via the CRP gives an equivalent collapsed representation:

$$p(x_{1:n}) = \int \left[\prod_{i=1}^{n} \int p(x_i \mid \theta)\, G(\mathrm{d}\theta) \right] \mathrm{dDP}(G).$$

Posterior inference (e.g. via collapsed Gibbs sampling) updates cluster assignments z_i and atom parameters ϕ_k jointly, allowing the number of clusters K_n to grow or shrink as evidence accumulates.

5.7 Simulation and Illustration

The following examples implement the three constructions above. The stick-breaking example directly codes the weights π_k defined in Section 5.4; the CRP example codes the sequential seating rule; the mixture example chains the two together. All three blocks are self-contained and can be run independently.

Stick-breaking construction. The `dp_stick` function below truncates the infinite sum to $K = 300$ atoms. The residual stick $1-\sum_{k=1}^{K}\pi_k$ is negligible for any practical α because $\mathbb{E}[\pi_k] \propto (1-1/\alpha)^k$ decreases geometrically.

```
set.seed(42)

dp_stick <- function(alpha, G_0_draw, K = 300) {
  v      <- rbeta(K, shape1 = 1, shape2 = alpha)
  pi     <- numeric(K)
  remain <- 1
  for (k in seq_len(K)) {
    pi[k]  <- remain * v[k]
    remain <- remain * (1 - v[k])
  }
  phi <- G_0_draw(K)
  list(pi = pi, phi = phi)
}

alphas <- c(1, 5, 25)
cols   <- c("steelblue", "firebrick", "darkgreen")
x_grid <- seq(-4, 4, length.out = 500)

par(mfrow = c(1, 3))
for (j in seq_along(alphas)) {
  dp <- dp_stick(
    alpha    = alphas[j],
    G_0_draw = function(k) rnorm(k, mean = 0, sd = 1)
  )

  ord   <- order(dp$phi)
  phi_s <- dp$phi[ord]
  cdf   <- cumsum(dp$pi[ord])

  plot(phi_s, cdf, type = "s", lwd = 1.5,
       col  = cols[j],
       xlim = c(-4, 4), ylim = c(0, 1),
       xlab = expression(theta),
       ylab = expression(G(theta)),
       main = paste0("DP(alpha = ", alphas[j], ", G0 = N(0,1))"))

  lines(x_grid, pnorm(x_grid),
```

```
        lty = 2, col = "grey50", lwd = 1.5)

  legend("topleft", bty = "n",
         legend = c("G  (DP draw)", "G0 = N(0,1)"),
         col    = c(cols[j], "grey50"),
         lty    = c(1, 2), lwd = 2)
}
par(mfrow = c(1, 1))
```

Remark 5.2. Each panel shows one realisation $G \sim \mathrm{DP}(\alpha, G_0)$ as a step-function CDF alongside the continuous base measure $G_0 = \mathrm{Normal}(0, 1)$ (dashed). As α increases, G places more, smaller atoms and its CDF tracks G_0 more closely, consistent with $\mathrm{Var}[G(A)] = G_0(A)(1-G_0(A))/(\alpha+1) \to 0$.

Chinese restaurant process. The code implements the seating rule directly: at step i, the probability vector is `c(tables, alpha) / (i - 1 + alpha)`, where `tables` holds the current size n_k of each occupied table, `alpha` is the weight on opening a new one, and the denominator $i-1+\alpha$ equals $n + \alpha$ with $n = i - 1$ customers already seated.

```
set.seed(7)

crp_sample <- function(n, alpha) {
  # tables: vector of current table sizes (n_k)
  # labels: cluster assignment z_i for each of n customers
  tables <- integer(0)
  labels <- integer(n)
  for (i in seq_len(n)) {
    K     <- length(tables)          # K_{i-1}: tables so far
    # P(z_i = k) = n_k / (i-1+alpha),  P(new) = alpha / (i-1+alpha)
    probs <- c(tables, alpha) / (i - 1 + alpha)
    z     <- sample(K + 1, size = 1, prob = probs)
    if (z == K + 1) {
      tables <- c(tables, 1L)        # open new table
    } else {
      tables[z] <- tables[z] + 1L   # join existing table
    }
    labels[i] <- z
  }
  list(labels = labels,
       sizes  = sort(tables, decreasing = TRUE),
       K      = length(tables))
}

n      <- 300
alphas <- c(0.5, 2, 10)
cols   <- c("steelblue", "firebrick", "darkgreen")

plot(NA, xlim = c(1, n), ylim = c(0, 45),
```

```
     xlab = "n  (customers)",
     ylab = "K_n  (tables occupied)",
     main = "CRP: cluster growth vs alpha * log(n)")

for (j in seq_along(alphas)) {
  labels  <- crp_sample(n, alphas[j])$labels
  # K_i = number of distinct tables after i customers
  K_trace <- sapply(seq_len(n),
                    function(i) length(unique(labels[1:i])))
  lines(seq_len(n), K_trace,
        col = cols[j], lwd = 1.8)
  # Theoretical expectation E[K_n] ~ alpha * log(n)
  lines(seq_len(n), alphas[j] * log(seq_len(n)),
        col = cols[j], lwd = 1, lty = 2)
}

legend("topleft", bty = "n",
       legend = c(paste0("alpha = ", alphas, "  (simulated)"),
                  "alpha * log(n)  (theory)"),
       col = c(cols, "grey40"),
       lty = c(1, 1, 1, 2), lwd = 2)
```

Remark 5.3. The solid lines are simulated cluster counts K_n; the dashed lines are the theoretical expectation $\alpha \log n$. Larger α produces more tables, but all three grow only logarithmically – the number of clusters is parsimonious relative to the number of observations n.

DP mixture model. This example implements the three-line generative model: draw G via stick-breaking, draw $\theta_i \sim G$ for $i = 1, \ldots, n$, then draw $x_i \mid \theta_i$. The `dp_stick` function is restated so the block runs independently.

```
set.seed(99)

dp_stick <- function(alpha, G_0_draw, K = 500) {
  v      <- rbeta(K, shape1 = 1, shape2 = alpha)
  pi     <- numeric(K)
  remain <- 1
  for (k in seq_len(K)) {
    pi[k]  <- remain * v[k]
    remain <- remain * (1 - v[k])
  }
  list(pi = pi, phi = G_0_draw(K))
}

# Generative model:
#   G ~ DP(alpha, N(0, sigma_0^2))
#   theta_i | G ~ G,            i = 1,...,n
#   x_i | theta_i ~ N(theta_i, sigma_x^2)
dp_mixture_sample <- function(n, alpha,
```

```
                                sigma_x = 0.5, sigma_0 = 3,
                                K = 500) {
  # Step 1: draw G via stick-breaking (truncated to K atoms)
  dp    <- dp_stick(alpha    = alpha,
                    G_0_draw = function(k) rnorm(k, 0, sigma_0),
                    K        = K)
  # Step 2: draw theta_i ~ G for i = 1,...,n
  idx   <- sample(K, size = n, replace = TRUE, prob = dp$pi)
  theta <- dp$phi[idx]
  # Step 3: draw x_i | theta_i ~ N(theta_i, sigma_x^2)
  x     <- rnorm(n, mean = theta, sd = sigma_x)
  list(x = x, theta = theta, clusters = idx)
}

n           <- 300
alphas      <- c(1, 5, 20)
cols        <- c("steelblue", "firebrick", "darkgreen")
sigma_0     <- 3;  sigma_x <- 0.5
marginal_sd <- sqrt(sigma_0^2 + sigma_x^2)
x_grid      <- seq(-10, 10, length.out = 300)

par(mfrow = c(1, 3))
for (j in seq_along(alphas)) {
  samp   <- dp_mixture_sample(n, alpha   = alphas[j],
                                sigma_x  = sigma_x,
                                sigma_0  = sigma_0)
  K_used <- length(unique(samp$clusters))

  plot(density(samp$x, bw = 0.4),
       col  = cols[j], lwd = 2,
       xlim = c(-10, 10), ylim = c(0, 0.2),
       main = paste0("alpha = ", alphas[j],
                     "  (", K_used, " clusters)"),
       xlab = "x", ylab = "Density")

  rug(samp$x, col = adjustcolor(cols[j], alpha.f = 0.3))

  # Marginal: x_i ~ N(0, sigma_0^2 + sigma_x^2)
  lines(x_grid, dnorm(x_grid, 0, marginal_sd),
        lty = 2, col = "grey50", lwd = 1.5)

  legend("topleft", bty = "n",
         legend = c("KDE of sample",
                    paste0("N(0, ", round(marginal_sd^2, 2), ")")),
         col = c(cols[j], "grey50"),
         lty = c(1, 2), lwd = 2)
}
par(mfrow = c(1, 1))
```

Remark 5.4. For small α the $n = 300$ draws concentrate on a few well-separated clusters; the kernel density estimate is clearly multimodal. As α increases the number of clusters K_n grows and the mixture approaches the marginal Normal$(0, \sigma_0^2 + \sigma_x^2)$ (dashed), obtained by integrating θ_i out against G_0.

6 Gaussian Processes: Random Functions for Regression

Dirichlet processes place priors over unknown *distributions*. Gaussian processes place priors over unknown *functions*. This section develops GP regression from first principles, covering kernel selection, posterior inference, hyperparameter learning, and implementation in R.

6.1 Motivation: Nonparametric Regression

Suppose we observe pairs $\{(x_i, y_i)\}_{i=1}^n$ generated by

$$y_i = f(x_i) + \varepsilon_i, \qquad \varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2).$$

A parametric approach assumes a fixed functional form for f , a polynomial of chosen degree, say, and the analyst must commit to model complexity before seeing the data. The GP approach instead places a prior *directly over the function*:

$$f \sim \text{GP}(m, k).$$

The posterior $p(f \mid \text{data})$ is then a distribution over functions that is updated by the observations, and model complexity is determined by the data through the marginal likelihood.

6.2 Definition and Formal Properties

Definition 6.1 (Gaussian process)**.** A *Gaussian process* is a collection of random variables $\{f(x) : x \in \mathcal{X}\}$ such that for any finite set of input points $x_1, \ldots, x_n \in \mathcal{X}$,

$$\big(f(x_1), \ldots, f(x_n)\big)^\top \sim \text{Normal}(\boldsymbol{m}, \boldsymbol{K}),$$

where $m_i = m(x_i)$ and $K_{ij} = k(x_i, x_j)$.

The GP is fully characterised by two functions.

- *Mean function* $m(x) = \mathbb{E}[f(x)]$: the prior expectation of f at x. Commonly set to zero.
- *Covariance (kernel) function* $k(x, x') = \text{Cov}(f(x), f(x'))$: encodes smoothness, periodicity, and length-scale assumptions.

Remark 6.2. The kernel k must be *positive semi-definite*: for any inputs $x_1, \ldots, x_n$ and scalars $c_1, \ldots, c_n$,

$$\sum_{i,j} c_i c_j \, k(x_i, x_j) \geq 0.$$

This guarantees $\boldsymbol{K}$ is a valid covariance matrix.

Table 3: Common covariance kernels for GP regression.

Kernel	**Formula** $k(x, x')$	**Property**
Squared exponential (SE)	$\sigma_f^2 \exp\left(-\dfrac{r^2}{2\ell^2}\right)$	Infinitely differentiable
Matérn $\nu = 3/2$	$\sigma_f^2\left(1 + \dfrac{\sqrt{3}\,r}{\ell}\right) e^{-\sqrt{3}r/\ell}$	Once differentiable
Matérn $\nu = 5/2$	$\sigma_f^2\left(1 + \dfrac{\sqrt{5}\,r}{\ell} + \dfrac{5r^2}{3\ell^2}\right) e^{-\sqrt{5}r/\ell}$	Twice differentiable
Periodic	$\sigma_f^2 \exp\left(-\dfrac{2\sin^2(\pi r/p)}{\ell^2}\right)$	Seasonal variation
Linear	$\sigma_f^2 (x - c)(x' - c)$	Bayesian linear regression

6.3 Common Kernel Functions

The kernel is the primary modelling choice in a GP. Table 3 summarises the most common options. Here $r = |x - x'|$, $\ell > 0$ is a length-scale, σ_f^2 is an output variance, and p is a period.

The hyperparameters ℓ and σ_f^2 control the qualitative character of the prior. A small ℓ allows rapid variation; a large ℓ enforces smoothness over longer input ranges. The SE kernel produces infinitely smooth functions, which is often unrealistically strong; the Matérn family is frequently preferred in practice.

Valid kernels may be combined to encode richer structure:

$$k_{\text{sum}}(x, x') = k_1(x, x') + k_2(x, x'),$$
$$k_{\text{product}}(x, x') = k_1(x, x') \cdot k_2(x, x').$$

For example, $k_{\text{SE}} + k_{\text{periodic}}$ captures a smooth long-term trend plus seasonal fluctuations.

6.4 GP Regression: Posterior Inference

6.4.1 Setup

Let $\boldsymbol{X} = (x_1, \ldots, x_n)^\top$ be training inputs, $\boldsymbol{y} = (y_1, \ldots, y_n)^\top$ be noisy observations, and $\boldsymbol{X}_* = (x_1^*, \ldots, x_m^*)^\top$ be test inputs. Define

$$\boldsymbol{K} = k(\boldsymbol{X}, \boldsymbol{X}) + \sigma^2 \boldsymbol{I} \in \mathbb{R}^{n \times n},$$
$$\boldsymbol{K}_* = k(\boldsymbol{X}_*, \boldsymbol{X}) \in \mathbb{R}^{m \times n},$$
$$\boldsymbol{K}_{**} = k(\boldsymbol{X}_*, \boldsymbol{X}_*) \in \mathbb{R}^{m \times m}.$$

6.4.2 Joint distribution

Under the prior $f \sim \text{GP}(0, k)$ and likelihood $\boldsymbol{y} \mid f \sim \text{Normal}(f(\boldsymbol{X}), \sigma^2 \boldsymbol{I})$, the joint distribution of training outputs and test function values is

$$\begin{pmatrix} \boldsymbol{y} \\ \mathbf{f}_* \end{pmatrix} \sim \text{Normal}\left(\mathbf{0},\ \begin{pmatrix} \boldsymbol{K} & \boldsymbol{K}_*^\top \\ \boldsymbol{K}_* & \boldsymbol{K}_{**} \end{pmatrix}\right).$$

6.4.3 Posterior predictive distribution

Conditioning on the observed data gives

$$\mathbf{f}_* \mid \boldsymbol{X}, \boldsymbol{y}, \boldsymbol{X}_* \;\sim\; \mathrm{Normal}\big(\bar{\mathbf{f}}_*,\; \mathrm{Cov}(\mathbf{f}_*)\big)\,, \tag{1}$$

where

$$\bar{\mathbf{f}}_* = \boldsymbol{K}_* \boldsymbol{K}^{-1} \boldsymbol{y}, \tag{2}$$

$$\mathrm{Cov}(\mathbf{f}_*) = \boldsymbol{K}_{**} - \boldsymbol{K}_* \boldsymbol{K}^{-1} \boldsymbol{K}_*^{\top}. \tag{3}$$

Remark 6.3 (Interpretation of (2)-(3))**.** The posterior mean (2) is a linear smoother of $\boldsymbol{y}$; as $\sigma^2 \to 0$ it interpolates the training points exactly. The posterior variance (3) shrinks near observed inputs and grows in unexplored regions, so uncertainty is quantified automatically without any additional procedure.

6.4.4 Computational cost

The dominant cost is the inversion (or Cholesky factorisation) of $\boldsymbol{K} \in \mathbb{R}^{n \times n}$, which requires $\mathcal{O}(n^3)$ time and $\mathcal{O}(n^2)$ memory. For large n, sparse or inducing-point approximations are required.

6.5 Marginal Likelihood and Hyperparameter Learning

6.5.1 The marginal likelihood

The hyperparameters $\boldsymbol{\psi} = (\ell, \sigma_f^2, \sigma^2)$ are typically unknown. Integrating out f yields the marginal (type-II) likelihood

$$p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\psi}) = \int p(\boldsymbol{y} \mid f, \sigma^2)\, p(f \mid \boldsymbol{\psi})\, \mathrm{d}f = \mathrm{Normal}(\boldsymbol{y} \mid \boldsymbol{0}, \boldsymbol{K}),$$

whose logarithm is

$$\log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\psi}) = \underbrace{-\tfrac{1}{2} \boldsymbol{y}^{\top} \boldsymbol{K}^{-1} \boldsymbol{y}}_{\text{data fit}} \quad \underbrace{-\tfrac{1}{2} \log |\boldsymbol{K}|}_{\text{complexity penalty}} \quad -\tfrac{n}{2} \log(2\pi).$$

The first term rewards kernels that explain the data well; the second penalises over-flexible models. The marginal likelihood therefore trades off fit against complexity automatically – a Bayesian form of model selection.

6.5.2 Optimisation

Maximising $\log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\psi})$ over $\boldsymbol{\psi}$ (empirical Bayes) yields estimates $\hat{\boldsymbol{\psi}}$. Gradients are available analytically:

$$\frac{\partial \log p}{\partial \psi_j} = \frac{1}{2} \operatorname{tr}\left[\left(\boldsymbol{\alpha}\boldsymbol{\alpha}^{\top} - \boldsymbol{K}^{-1} \right) \frac{\partial \boldsymbol{K}}{\partial \psi_j} \right], \qquad \boldsymbol{\alpha} = \boldsymbol{K}^{-1} \boldsymbol{y}.$$

Standard optimisers (L-BFGS, Adam) are used in practice.

6.6 Why Is GP Bayesian Nonparametric?

The GP prior over $f(\cdot)$ lives in an infinite-dimensional function space $\mathcal{F}_\infty$, yet inference remains analytically tractable because every finite-dimensional marginal is multivariate Gaussian. The effective complexity, the number of "active" degrees of freedom, grows with the data through the marginal likelihood, rather than being fixed in advance.

6.7 Theoretical Properties

6.7.1 RKHS connection

Every kernel k generates a reproducing kernel Hilbert space (RKHS) $\mathcal{H}_k$. The GP posterior mean is the minimum norm element of $\mathcal{H}_k$ consistent with the data:

$$\bar{f}_* = \text{argmin}_{f \in \mathcal{H}_k} \|f\|^2_{\mathcal{H}_k} \quad \text{subject to} \quad (y_i - f(x_i)) \approx 0.$$

This connects GP regression to kernel ridge regression (KRR): the posterior mean equals the KRR solution with regularisation parameter $\lambda = \sigma^2$.

6.7.2 Posterior contraction

Let f^* be the true data-generating function. Under regularity conditions on f^* and the kernel, the posterior contracts around f^* at the minimax-optimal rate:

$$\Pi(\|f - f^*\|_n > M_n \varepsilon_n \mid y_{1:n}) \to 0$$

in probability, where ε_n is the minimax rate for functions in $\mathcal{H}_k$ and $M_n \to \infty$ arbitrarily slowly. In particular, the GP does not overfit despite placing a prior on an infinite-dimensional space.

6.7.3 Bias-variance interpretation

The posterior mean is a linear smoother,

$$\hat{f}(x^*) = \boldsymbol{k}_*^\top (\boldsymbol{K} + \sigma^2 \boldsymbol{I})^{-1} \boldsymbol{y}, \qquad \boldsymbol{k}_* = k(\boldsymbol{X}, x^*).$$

As $\sigma^2 \to 0$ the smoother interpolates (zero bias, high variance); as $\sigma^2 \to \infty$ it collapses to the prior mean (high bias, zero variance).

6.8 GP Regression in R

The following examples progress from a convenience-function interface to a full from-scratch implementation, then demonstrate hyperparameter optimisation and kernel comparison.

6.8.1 Example 1: `kernlab::gausspr`

```
library(kernlab)   # install.packages("kernlab")

# Simulate data
set.seed(42)
n <- 60
x <- sort(runif(n, 0, 2 * pi))
f <- function(x) sin(x) + 0.5 * sin(2 * x)
y <- f(x) + rnorm(n, sd = 0.3)

# Fit GP with squared-exponential (RBF) kernel
# Note: kernlab uses sigma = 1 / (2 * ell^2)
gp_fit <- gausspr(
  x      = x,
  y      = y,
```

```
  kernel = "rbfdot",
  kpar   = list(sigma = 0.5),
  var    = 0.09              # noise variance sigma^2
)

# Predict and plot
x_star <- seq(0, 2 * pi, length.out = 200)
mu_hat <- predict(gp_fit, x_star)

plot(x, y, pch = 20, col = "steelblue", cex = 0.9,
     xlab = "x", ylab = "y",
     main = "GP regression (kernlab)")
lines(x_star, f(x_star), col = "black",     lwd = 2, lty = 2)
lines(x_star, mu_hat,    col = "firebrick", lwd = 2)
legend("topright", bty = "n",
       legend = c("Observations", "True f", "Posterior mean"),
       col    = c("steelblue", "black", "firebrick"),
       lty    = c(NA, 2, 1), pch = c(20, NA, NA))
```

Remark 6.4. `kernlab` parameterises the RBF kernel as $k(x, x') = \exp(-\sigma\, r^2)$, so `sigma`$= 1/(2\ell^2)$. Larger `sigma` corresponds to smaller ℓ and faster variation.

6.8.2 Example 2: Posterior mean and uncertainty from scratch

This example implements equations (2)-(3) directly, making the linear algebra explicit.

```
# Squared-exponential kernel matrix
se_kernel <- function(x1, x2, ell = 1, sf2 = 1) {
  r2 <- outer(x1, x2, FUN = function(a, b) (a - b)^2)
  sf2 * exp(-r2 / (2 * ell^2))
}

# Data
set.seed(7)
n     <- 40
x     <- sort(runif(n, 0, 5))
f_true <- function(x) x * sin(x)
y     <- f_true(x) + rnorm(n, sd = 0.5)

# Fixed hyperparameters (for illustration)
ell <- 1.5;  sf2 <- 4.0;  sn2 <- 0.25

# Covariance matrices
x_star <- seq(0, 5, length.out = 300)
K   <- se_kernel(x, x, ell, sf2) + sn2 * diag(n)
Ks  <- se_kernel(x_star, x, ell, sf2)
Kss <- se_kernel(x_star, x_star, ell, sf2)

# Posterior via Cholesky (numerically stable)
```

```
L     <- t(chol(K))                                      # lower factor
alpha <- backsolve(t(L), forwardsolve(L, y))             # K^{-1} y
mu_star    <- Ks %*% alpha                               # posterior mean
v          <- forwardsolve(L, t(Ks))
Sigma_star <- Kss - t(v) %*% v                          # posterior cov
sd_star    <- sqrt(pmax(diag(Sigma_star), 0))

# Plot
plot(x, y, pch = 19, col = "steelblue", cex = 0.8,
     ylim = range(c(y, mu_star + 2*sd_star, mu_star - 2*sd_star)),
     xlab = "x", ylab = "f(x)",
     main = "GP posterior mean and 95% credible band")
polygon(c(x_star, rev(x_star)),
        c(mu_star + 1.96*sd_star, rev(mu_star - 1.96*sd_star)),
        col = adjustcolor("firebrick", 0.15), border = NA)
lines(x_star, mu_star,      col = "firebrick", lwd = 2.5)
lines(x_star, f_true(x_star), col = "black",  lwd = 1.5, lty = 2)
```

Remark 6.5. Solving via the Cholesky factorisation $\boldsymbol{K} = \boldsymbol{L}\boldsymbol{L}^\top$ computes $\boldsymbol{K}^{-1}\boldsymbol{y}$ as $\boldsymbol{L}^{-\top}(\boldsymbol{L}^{-1}\boldsymbol{y})$ at cost $\mathcal{O}(n^3)$ once, then $\mathcal{O}(n^2)$ per test point, and avoids the numerical issues that arise from explicit inversion.

6.8.3 Example 3: Sampling prior and posterior paths

```
set.seed(2024)
n_samps <- 6

# Prior samples
K_prior  <- se_kernel(x_star, x_star, 1.5, 4) + 1e-8*diag(300)
L_prior  <- t(chol(K_prior))
prior_samples <- L_prior %*% matrix(rnorm(300 * n_samps), 300)

# Posterior samples (shift by posterior mean)
L_post       <- t(chol(Sigma_star + 1e-8*diag(300)))
post_samples <- sweep(
  L_post %*% matrix(rnorm(300 * n_samps), 300),
  1, mu_star, "+"
)

par(mfrow = c(1, 2))
matplot(x_star, prior_samples, type = "l", lty = 1,
        col = adjustcolor(1:n_samps, 0.7),
        xlab = "x", ylab = "f(x)",
        main = "Prior sample paths")
abline(h = 0, lty = 2, col = "gray70")

matplot(x_star, post_samples, type = "l", lty = 1,
        col = adjustcolor(1:n_samps, 0.7),
```

```
        xlab = "x", ylab = "f(x)",
        main = "Posterior sample paths")
points(x, y, pch = 19, col = "steelblue", cex = 0.9)
par(mfrow = c(1, 1))
```

6.8.4 Example 4: Hyperparameter optimisation via marginal likelihood

```
log_mlik_neg <- function(params, x, y) {
  ell <- exp(params[1]);  sf2 <- exp(params[2]);  sn2 <- exp(params[3])
  n   <- length(y)
  K   <- se_kernel(x, x, ell, sf2) + sn2 * diag(n)
  L   <- tryCatch(t(chol(K)), error = function(e) NULL)
  if (is.null(L)) return(Inf)
  alpha    <- backsolve(t(L), forwardsolve(L, y))
  log_det  <- 2 * sum(log(diag(L)))
  lml      <- -0.5 * (t(y) %*% alpha) - 0.5 * log_det -
              (n/2) * log(2*pi)
  -as.numeric(lml)   # return negative for minimisation
}

opt <- optim(
  par     = c(log(1.5), log(4.0), log(0.25)),
  fn      = log_mlik_neg,
  x       = x, y = y,
  method  = "L-BFGS-B",
  control = list(maxit = 500)
)

ell_hat <- exp(opt$par[1])
sf2_hat <- exp(opt$par[2])
sn2_hat <- exp(opt$par[3])
cat(sprintf("ell = %.3f,  sf2 = %.3f,  sn2 = %.3f\n",
            ell_hat, sf2_hat, sn2_hat))

# Recompute posterior with learned hyperparameters
K_opt   <- se_kernel(x, x, ell_hat, sf2_hat) + sn2_hat * diag(n)
Ks_opt  <- se_kernel(x_star, x, ell_hat, sf2_hat)
Kss_opt <- se_kernel(x_star, x_star, ell_hat, sf2_hat)
L_opt   <- t(chol(K_opt))
alpha_opt <- backsolve(t(L_opt), forwardsolve(L_opt, y))
mu_opt    <- Ks_opt %*% alpha_opt
v_opt     <- forwardsolve(L_opt, t(Ks_opt))
sd_opt    <- sqrt(pmax(diag(Kss_opt - t(v_opt) %*% v_opt), 0))

plot(x, y, pch = 19, col = "steelblue", cex = 0.8,
     xlab = "x", ylab = "f(x)",
     main = "GP with optimised hyperparameters")
polygon(c(x_star, rev(x_star)),
```

```
        c(mu_opt + 1.96*sd_opt, rev(mu_opt - 1.96*sd_opt)),
        col = adjustcolor("darkorange", 0.2), border = NA)
lines(x_star, mu_opt,          col = "darkorange", lwd = 2.5)
lines(x_star, f_true(x_star),  col = "black",      lwd = 1.5, lty = 2)
```

6.8.5 Example 5: `GPfit` package

```
library(GPfit)   # install.packages("GPfit")

set.seed(99)
n     <- 50
x_raw <- sort(runif(n, 0, 2*pi))
y_raw <- sin(x_raw) + rnorm(n, sd = 0.2)

# GPfit requires inputs in [0, 1]
x_sc <- (x_raw - min(x_raw)) / diff(range(x_raw))

gp2  <- GP_fit(X = matrix(x_sc, ncol = 1), Y = y_raw)
print(gp2)   # estimated length-scale (beta) and variance

x_star_raw <- seq(0, 2*pi, length.out = 300)
x_star_sc  <- (x_star_raw - min(x_raw)) / diff(range(x_raw))

pred <- predict.GP(gp2, xnew = matrix(x_star_sc, ncol = 1))
mu2  <- pred$Y_hat
sd2  <- sqrt(pred$MSE)

plot(x_raw, y_raw, pch = 19, col = "steelblue", cex = 0.8,
     xlab = "x", ylab = "y", main = "GP regression via GPfit")
polygon(c(x_star_raw, rev(x_star_raw)),
        c(mu2 + 1.96*sd2, rev(mu2 - 1.96*sd2)),
        col = adjustcolor("purple", 0.15), border = NA)
lines(x_star_raw, mu2,             col = "purple", lwd = 2)
lines(x_star_raw, sin(x_star_raw), col = "black",  lwd = 1.5, lty = 2)
```

6.8.6 Example 6: Kernel comparison

```
library(kernlab)
set.seed(11)

f_true2 <- function(x) 0.5*x + 2*sin(x)
n  <- 60
x  <- sort(runif(n, 0, 3*pi))
y  <- f_true2(x) + rnorm(n, sd = 0.4)
xs <- seq(0, 3*pi, length.out = 300)

gp_se  <- gausspr(x, y, kernel = "rbfdot",
                  kpar = list(sigma = 0.3))
```

```
gp_mat <- gausspr(x, y, kernel = "maternVVdot",
                  kpar = list(nu = 2.5))
gp_pol <- gausspr(x, y, kernel = "polydot",
                  kpar = list(degree = 3, scale = 1, offset = 1))

plot(x, y, pch = 19, col = "gray50", cex = 0.8,
     xlab = "x", ylab = "y", main = "Kernel comparison")
lines(xs, f_true2(xs),        col = "black",     lwd = 2, lty = 2)
lines(xs, predict(gp_se,  xs), col = "firebrick", lwd = 2)
lines(xs, predict(gp_mat, xs), col = "steelblue", lwd = 2)
lines(xs, predict(gp_pol, xs), col = "darkgreen", lwd = 2)
legend("topleft", bty = "n",
       legend = c("True f", "SE", "Matern 5/2", "Polynomial (d=3)"),
       col    = c("black", "firebrick", "steelblue", "darkgreen"),
       lty    = c(2, 1, 1, 1), lwd = 2)
```

7 Feature Allocation: Indian Buffet Process

Clustering assigns each observation to exactly one latent group. This is appropriate when groups are mutually exclusive, but many settings require a richer structure: a single observation may possess *multiple* latent features simultaneously. A film, for instance, may be simultaneously action, comedy, and romance; a patient may exhibit several co-occurring syndromes; a document may belong to several topics.

The goal is therefore to infer a binary *feature matrix*

$$\boldsymbol{Z} \in \{0,1\}^{n\times K}, \qquad Z_{ik} = \mathbf{1}[\text{observation } i \text{ possesses feature } k],$$

where K is not fixed in advance. The Indian buffet process (IBP) is the canonical BNP prior for this setting.

The contrast with the Dirichlet process is clean:

- the DP induces *clustering*, each observation receives one latent label;
- the IBP induces *feature allocation*, each observation receives a binary vector of latent labels, and the number of features grows with the data.

7.1 From Beta-Bernoulli to the IBP

It is instructive to arrive at the IBP as the infinite limit of a finite feature model, mirroring the derivation of the DP from a finite Dirichlet mixture.

Suppose there are K features. Place independent Beta priors on the feature prevalences and Bernoulli likelihoods on the indicators:

$$\pi_k \overset{\text{iid}}{\sim} \text{Beta}\big(\tfrac{\alpha}{K}, 1\big), \qquad Z_{ik} \mid \pi_k \overset{\text{ind}}{\sim} \text{Bernoulli}(\pi_k), \quad k = 1, \dots, K.$$

Integrating out π_k, the probability that a specific set $S \subseteq \{1, \dots, n\}$ of observations possesses feature k is

$$P\big(\{i : Z_{ik} = 1\} = S\big) = \frac{\Gamma(\alpha/K + |S|)\,\Gamma(1 + n - |S|)}{\Gamma(\alpha/K)\,\Gamma(1)\,\Gamma(\alpha/K + 1 + n)}.$$

Taking $K \to \infty$ with α fixed yields the IBP: the number of features an observation possesses remains finite in expectation (equal to αH_n, where H_n is the n-th harmonic number), while the total number of features grows as $\alpha \log n$.

7.2 The Indian Buffet Process

The IBP has an evocative sequential construction. Imagine n customers entering a buffet with infinitely many dishes arranged in a line.

- **Customer 1** samples dishes from the left, trying each independently with probability $\alpha/(\alpha+1)$, then tries Poisson(α) new dishes beyond those already sampled.
- **Customer** i ($i \geq 2$), having observed the choices of the preceding $i-1$ customers, tries each previously sampled dish k with probability m_k/i, where m_k is the number of earlier customers who tried dish k. After going through all existing dishes, customer i tries Poisson(α/i) new dishes.

Equivalently, the probability that customer $n+1$ selects dish k given the choices $\boldsymbol{Z}_{1:n}$ of the first n customers is

$$P(Z_{n+1,k} = 1 \mid \boldsymbol{Z}_{1:n}) = \frac{m_k}{n+1},$$

and the number of new dishes tried by customer $n+1$ is drawn from Poisson$\big(\alpha/(n+1)\big)$.

Remark 7.1 (Exchangeability)**.** The IBP is *exchangeable* in the sense that the distribution of the binary matrix $\boldsymbol{Z}$ (up to column reordering) does not depend on the order in which customers arrive. This is the analogue of the CRP's exchangeability property for clustering.

7.3 Properties of the IBP

Let K_n denote the total number of features used by n observations, and let K_i^+ denote the number of new features introduced by observation i.

- *Feature counts.* $K_i^+ \sim \text{Poisson}(\alpha/i)$, so

$$\mathbb{E}[K_n] = \alpha \sum_{i=1}^{n} \frac{1}{i} = \alpha H_n \approx \alpha \log n.$$

  The number of features grows logarithmically with the data, as in the CRP.

- *Sparsity.* The expected number of features possessed by a single observation is α, regardless of n. The feature matrix becomes increasingly sparse as n grows.
- *Role of* α. Large α produces richer representations with more features per observation; small α yields sparser matrices.

7.4 The Beta Process: Measure-Theoretic Foundation

Just as the DP has a measure-theoretic definition as a random probability measure, the IBP is the marginal process obtained by integrating out a *beta process* prior $B \sim \text{BP}(\alpha, B_0)$ on a completely

random measure. Feature prevalences π_k correspond to the atoms of B, and the binary indicators Z_{ik} are Bernoulli draws with those atoms as probabilities:

$$B \sim \mathrm{BP}(\alpha, B_0), \qquad Z_{ik} \mid B \overset{\text{ind}}{\sim} \mathrm{Bernoulli}(\pi_k), \quad \pi_k \sim B.$$

Marginalising over B recovers the IBP sequential construction. The beta process thus plays the same role for feature allocation that the Dirichlet process plays for clustering.

7.5 Latent Feature Model

In a *linear Gaussian latent feature model*, the IBP prior on $\boldsymbol{Z}$ is combined with a Gaussian likelihood:

$$\begin{aligned} \boldsymbol{Z} &\sim \mathrm{IBP}(\alpha), \\ \boldsymbol{A}_k &\overset{\text{iid}}{\sim} \mathrm{Normal}(\mathbf{0}, \sigma_A^2 \boldsymbol{I}), \quad k = 1, 2, \ldots, \\ \boldsymbol{x}_i \mid \boldsymbol{Z}, \boldsymbol{A} &\sim \mathrm{Normal}\left(\sum_k Z_{ik}\, \boldsymbol{A}_k,\ \sigma_x^2 \boldsymbol{I} \right). \end{aligned}$$

Here $\boldsymbol{A}_k \in \mathbb{R}^D$ is the k-th latent feature vector and $\boldsymbol{x}_i \in \mathbb{R}^D$ is the observed data vector for observation i. In matrix form,

$$\boldsymbol{X} \mid \boldsymbol{Z}, \boldsymbol{A} \ \sim\ \mathrm{Normal}(\boldsymbol{Z}\boldsymbol{A},\ \sigma_x^2 \boldsymbol{I}),$$

so $\boldsymbol{Z}\boldsymbol{A}$ is a low-rank factorisation of $\boldsymbol{X}$ with a nonparametric prior on the number of factors. Posterior inference over $\boldsymbol{Z}$ and $\boldsymbol{A}$ is typically performed via collapsed Gibbs sampling or variational inference.

7.6 IBP in R: Simulation

The following code simulates a binary feature matrix from the IBP sequential construction and visualises the result.

```
set.seed(42)

ibp_sample <- function(n, alpha) {
  # Returns a binary matrix Z (n x K) sampled from IBP(alpha)
  Z   <- matrix(0, nrow = n, ncol = 0)   # start with no features
  m   <- integer(0)                      # dish counts

  for (i in seq_len(n)) {
    K_cur <- ncol(Z)

    # Sample existing dishes
    if (K_cur > 0) {
      probs      <- m / i
      new_row    <- rbinom(K_cur, 1, probs)
      Z[i, ]     <- new_row
      m          <- m + new_row
    }
```

```
    # Sample new dishes
    K_new <- rpois(1, alpha / i)
    if (K_new > 0) {
      Z <- cbind(Z, matrix(0, nrow = n, ncol = K_new))
      Z[i, (K_cur + 1):(K_cur + K_new)] <- 1
      m <- c(m, rep(1, K_new))
    }
  }
  Z
}

# Simulate and plot
n     <- 50
alpha <- 3
Z     <- ibp_sample(n, alpha)

# Remove all-zero columns (never chosen)
Z <- Z[, colSums(Z) > 0, drop = FALSE]

cat(sprintf("n = %d observations,  K = %d features used\n",
            n, ncol(Z)))
cat(sprintf("Expected features: alpha * H_n = %.2f\n",
            alpha * sum(1 / seq_len(n))))

# Visualise
image(t(Z[nrow(Z):1, ]), axes = FALSE,
      col  = c("white", "steelblue"),
      main = sprintf("IBP feature matrix  (alpha = %g)", alpha),
      xlab = "Feature", ylab = "Observation")
box()
```

8 Beyond the Dirichlet Process: Pitman-Yor Process

The Dirichlet process produces cluster sizes that decay *exponentially*: the k-th largest cluster has expected size of order e^{-ck} for some constant $c > 0$. Many real-world phenomena, word frequencies in natural language, allele frequencies in population genetics, degree distributions in networks, exhibit *power-law* behaviour instead, where the k-th largest cluster has size of order $k^{-1/d}$ for some $d \in (0, 1)$. The Pitman-Yor process is the natural two-parameter generalisation of the DP that accommodates this.

8.1 Definition

Definition 8.1 (Pitman-Yor process)**.** A random probability measure G follows a *Pitman-Yor process* with discount parameter $d \in [0, 1)$, concentration parameter $\alpha > -d$, and base measure G_0,

written $G \sim Pittman - Yor(d, \alpha, G_0)$, if it admits the stick-breaking representation

$$G = \sum_{k=1}^{\infty} \pi_k \, \delta_{\phi_k}, \qquad \phi_k \overset{\text{iid}}{\sim} G_0,$$

where the weights are constructed as

$$\pi_k = v_k \prod_{j=1}^{k-1} (1 - v_j), \qquad v_k \overset{\text{ind}}{\sim} \text{Beta}(1 - d,\ \alpha + kd).$$

Setting $d = 0$ recovers the DP: the Beta variates reduce to $\text{Beta}(1, \alpha)$, the original stick-breaking construction of Sethuraman [1994]. For $d > 0$ the later sticks are broken more generously, allowing more atoms to carry non-negligible weight and producing heavier tails.

8.2 Two-Parameter Chinese Restaurant Process

As with the DP, marginalising out G gives a sequential assignment rule. The *two-parameter CRP* (Pitman 1995) assigns customer $n + 1$ to tables as follows:

$$P(z_{n+1} = k \mid z_{1:n}) = \begin{cases} \dfrac{n_k - d}{\alpha + n}, & \text{existing table } k \text{ with } n_k > 0, \\ \dfrac{\alpha + d\, K_n}{\alpha + n}, & \text{a new table,} \end{cases}$$

where n_k is the number of customers at table k and K_n is the current number of occupied tables. Setting $d = 0$ recovers the standard CRP exactly.

The discount d penalises large tables: the effective attraction of table k is $n_k - d$ rather than n_k, so mass is redistributed toward new tables relative to the DP.

8.3 Power-Law Cluster Growth

The two parameters govern cluster growth in qualitatively different ways.

- *Number of clusters.*

$$\mathbb{E}[K_n] \ \approx \ \frac{\Gamma(\alpha + 1 + d)}{\Gamma(\alpha + 1)} \cdot \frac{n^d}{\Gamma(1 + d)}, \quad n \to \infty.$$

  For $d = 0$ this reduces to $\alpha \log n$ (DP). For $d > 0$ the number of clusters grows as a *power law* n^d, so the PY process generates many more non-trivial clusters than the DP for large n.

- *Cluster-size distribution.* Under the DP, the probability that a uniformly chosen observation belongs to a cluster of size s decays exponentially in s. Under the PY process with discount d, this probability decays as $s^{-(1+d)}$, a proper power law.

- *Role of α.* Holding d fixed, larger α produces more clusters of smaller average size, as in the DP.

Table 4: Comparison of DP and Pitman-Yor process.

	Dirichlet process	**Pitman-Yor process**
Parameters	$\alpha > 0$	$d \in [0, 1)$, $\alpha > -d$
Stick-breaking	$v_k \sim \text{Beta}(1, \alpha)$	$v_k \sim \text{Beta}(1 - d,\, \alpha + kd)$
Cluster growth	$\mathbb{E}[K_n] \sim \alpha \log n$	$\mathbb{E}[K_n] \sim C(\alpha, d)\, n^d$
Cluster sizes	Exponential tails	Power-law tails $\sim s^{-(1+d)}$
CRP attraction	$n_k/(\alpha + n)$	$(n_k - d)/(\alpha + n)$
Special case	$d = 0$	Generalises DP

8.4 Applications

The power-law property makes the Pitman-Yor process the natural prior in several domains.

- *Language modelling.* Word frequencies in natural language follow Zipf's law: the r-th most frequent word has frequency $\propto r^{-1}$. Hierarchical PY language models [**?**] place a PY process at each context level, yielding distributions over word sequences with the correct power-law tails.
- *Population genetics.* Allele frequencies in a large population follow Ewens' sampling formula when $d = 0$. The PY extension with $d > 0$ accommodates more realistic allele-frequency spectra under models with varying population size.
- *Network clustering.* Community sizes in large social or biological networks are empirically heavy-tailed. The PY process provides a principled prior over the number and sizes of communities without fixing either in advance.

8.5 Simulation in R

```
set.seed(42)

# Stick-breaking weights for PY(d, alpha, G_0)
# Returns pi_1, ..., pi_K such that sum >= 1 - tol
py_weights <- function(alpha, d, tol = 1e-4, K_max = 2000) {
  pis    <- numeric(K_max)
  remain <- 1
  for (k in seq_len(K_max)) {
    v          <- rbeta(1, 1 - d, alpha + k * d)
    pis[k]     <- remain * v
    remain     <- remain * (1 - v)
    if (remain < tol) break
  }
  pis[pis > 0]
}

# Simulate cluster assignments for n observations
simulate_crp <- function(n, alpha, d) {
  tables <- integer(0)   # table sizes
  labels <- integer(n)
```

```
  for (i in seq_len(n)) {
    K <- length(tables)
    probs <- if (K == 0) 1 else
      c((tables - d) / (alpha + i - 1),
        (alpha + d * K) / (alpha + i - 1))
    choice <- sample(K + 1, 1, prob = probs)
    if (choice == K + 1) {
      tables  <- c(tables, 1L)
    } else {
      tables[choice] <- tables[choice] + 1L
    }
    labels[i] <- choice
  }
  list(labels = labels, sizes = sort(tables, decreasing = TRUE))
}

# Compare DP (d=0) and PY (d=0.5) with same alpha
n     <- 500
alpha <- 5

dp_out <- simulate_crp(n, alpha, d = 0.0)
py_out <- simulate_crp(n, alpha, d = 0.5)

cat(sprintf("DP  (d=0.0): %d clusters\n", length(dp_out$sizes)))
cat(sprintf("PY  (d=0.5): %d clusters\n", length(py_out$sizes)))

# Plot cluster-size rank curves on log-log scale
par(mfrow = c(1, 2))

plot(seq_along(dp_out$sizes), dp_out$sizes,
     log  = "xy", type = "b", pch = 19, cex = 0.6,
     col  = "steelblue",
     xlab = "Rank", ylab = "Cluster size",
     main = "DP  (d = 0): exponential decay")

plot(seq_along(py_out$sizes), py_out$sizes,
     log  = "xy", type = "b", pch = 19, cex = 0.6,
     col  = "firebrick",
     xlab = "Rank", ylab = "Cluster size",
     main = "PY  (d = 0.5): power-law decay")

par(mfrow = c(1, 1))

# Overlay stick-breaking weight distributions
pi_dp <- py_weights(alpha, d = 0.0)
pi_py <- py_weights(alpha, d = 0.5)

plot(seq_along(pi_dp), pi_dp,
```

```
     type = "h", lwd = 1.5, col = "steelblue",
     xlim = c(1, max(length(pi_dp), length(pi_py))),
     ylim = c(0, max(pi_dp, pi_py)),
     xlab = "Atom index k", ylab = expression(pi[k]),
     main = "Stick-breaking weights: DP vs PY")
lines(seq_along(pi_py), pi_py,
      type = "h", lwd = 1.5, col = "firebrick")
legend("topright", bty = "n",
       legend = c("DP  (d = 0)", "PY  (d = 0.5)"),
       col    = c("steelblue", "firebrick"), lwd = 2)
```

9 Why BNP Works: Consistency and Adaptivity

A natural concern about BNP models is whether placing a prior over an infinite-dimensional space leads to sensible behaviour: does the posterior ever settle on an answer, or does it remain hopelessly uncertain regardless of how much data are collected? The answer is reassuring on both counts.

The posterior concentrates. As the sample size grows, the posterior mass piles up around the true data-generating object $G^\star$ and spreads less and less. In the limit,

$$p(G \mid X_{1:n}) \;\to\; \delta_{G^\star} \quad \text{as } n \to \infty.$$

The only requirement is that the prior assigns positive probability to neighbourhoods of $G^\star$, a mild condition satisfied whenever the base measure G_0 has sufficiently broad support.

The posterior converges at the right speed. Not only does the posterior converge, it does so at the fastest rate the problem allows. For smooth true densities or functions, BNP posteriors achieve the same minimax-optimal contraction rates as the best classical nonparametric estimators, without the analyst needing to specify the smoothness class in advance.

Complexity adapts to the data. A classical model with p fixed parameters cannot adapt: too few parameters incur bias, too many incur variance, and the right choice depends on an unknown property of the truth. BNP models sidestep this entirely. The effective number of active components, clusters in a DP mixture, latent features in an IBP model, degrees of freedom in a GP, grow with the sample size at exactly the rate the data support. Unnecessary complexity is suppressed automatically by the prior acting as a regulariser.

Uncertainty is always quantified. Credible intervals and posterior bands for any quantity of interest follow directly from the posterior, without separate bootstrap or approximation procedures. Uncertainty widens where data are sparse and narrows where they are dense – behaviour that is automatic in the Bayesian framework but requires explicit additional work in frequentist nonparametrics.

The unifying message is simple: BNP models do not overfit despite their infinite-dimensional parameter spaces. The prior concentrates posterior mass on parsimonious explanations, complexity grows only as the evidence demands, and the analyst never needs to specify model size in advance.

Table 5: Theoretical guarantees for common BNP models.

Model	**Consistent?**	**Rate**	**Adaptive?**
DP mixture	Yes	$n^{-s/(2s+1)}$ (minimax)	Yes
GP regression	Yes	Minimax for $\mathcal{H}_k$	Yes
IBP latent feature	Yes	Problem-dependent	Yes
Pitman-Yor mixture	Yes	$n^{-s/(2s+1)}$ (minimax)	Yes

Concise Summary

- **Bayesian inference** updates a prior $P(\theta)$ via Bayes' theorem to obtain the posterior $P(\theta \mid \boldsymbol{y}) \propto P(\theta)\, P(\boldsymbol{y} \mid \theta)$, yielding a full distribution over θ rather than a point estimate.
- **Bayesian nonparametrics** extends this to infinite-dimensional parameters: distributions, functions, partitions, and feature matrices. Model complexity is not fixed in advance but learned from the data.
- **Dirichlet process** $G \sim \mathrm{DP}(\alpha, G_0)$ is a prior over probability measures. Draws are almost surely discrete, inducing clustering. The marginal process is the CRP; cluster count grows as $\alpha \log n$.
- **Gaussian process** $f \sim \mathrm{GP}(m, k)$ is a prior over functions. The posterior is analytically tractable (multivariate Gaussian) and provides automatic uncertainty quantification. Hyperparameters are learned by maximising the marginal likelihood.
- **Indian buffet process** is a prior over infinite binary feature matrices, allowing each observation to possess multiple latent features. It arises as the infinite limit of a Beta-Bernoulli model; expected feature count grows as $\alpha \log n$.
- **Pitman-Yor process** $G \sim Pittman - Yor(d, \alpha, G_0)$ generalises the DP with a discount parameter $d \in [0, 1)$. Cluster sizes follow a power law $\sim k^{-1/d}$ and cluster count grows as n^d, making it suitable for language, genetics, and network applications.
- **Consistency and adaptivity.** BNP posteriors contract to the truth at minimax-optimal rates without the analyst specifying smoothness or complexity. Uncertainty is quantified automatically throughout.

  BNP replaces fixed model complexity with uncertainty over complexity, learned directly from the data.

References

James O Berger and Robert L Wolpert. The likelihood principle and generalizations. In *The likelihood principle*, volume 6, pages 19–65. Institute of Mathematical Statistics, 1988.

Jayaram Sethuraman. A constructive definition of dirichlet priors. *Statistica sinica*, pages 639–650, 1994.